

The Unsolved Heart of Computing: P vs NP and Its Implications

Jay Chen

CUNY College of Staten Island

CSC 490 Seminar in Computer Science

Lou Lacona

Abstract

The P vs NP problem is one of the most significant open questions in computer science and mathematics. Designated as one of the seven Millennium Prize Problems by the Clay Mathematics Institute, it asks whether every problem whose solution can be verified quickly (in polynomial time) can also be solved quickly. This deceptively simple question holds profound implications for cryptography, artificial intelligence, biology, logistics, and theoretical mathematics. If solved, the result could revolutionize or destabilize the foundations of digital security and computation. This paper explores the history of the P vs NP problem, explains its theoretical foundations, and discusses the implications of both potential outcomes: $P = NP$ and $P \neq NP$.

Historical Background

The roots of the P vs NP problem trace back to a landmark paper by Stephen Cook in 1971 titled "The Complexity of Theorem-Proving Procedures". In this paper, Cook introduced the concept of NP-completeness and formally defined the P vs NP question. This paper revolutionized the field known as Computational complexity theory. For his foundational contributions, he was later awarded the Turing Award. In 1972, Richard Karp extended Cook's work by identifying 21 problems that are NP-complete, further emphasizing the centrality and difficulty of these computational challenges. Since then, P vs NP has remained a focal point in theoretical computer science. Listed among the seven Millennium Prize Problems by the Clay Mathematics Institute (CMI), it challenges our understanding of what is computationally feasible. The CMI, a private, non-profit organization dedicated to advancing mathematical knowledge, offers a \$1 million prize for a correct solution to any of these problems. The P vs NP

question lies at the heart of complexity theory and has vast implications across science, technology, and society.

Understanding P vs NP

It might be a bit confusing at first as to what the P vs NP asks. To help understand this question, we must go through some prerequisite knowledge, mainly in algorithmic time complexity. From there, we will understand P and NP as complexity classes and explain what the question is truly asking. In computer science, problems are solved via algorithms. Algorithms are just steps to solving a problem. Different algorithms take more time than others, and we separate these differences in time into different time complexities. Take, for example, the problem of finding a number from a list of numbers, formally called searching. You can look for that number by linearly looking through each element until you find that number. This is a linear time complexity and is formally written as $O(n)$. A faster approach to finding a number in a list is an algorithm called binary search, where, given a sorted list, you look at the first half of the list, and if the number comes after the element you look in the second half, if it comes before you look in the smaller half. You repeat this, and each time you divide the list in half until you find your number, thereby cutting the search space in half. This is a faster approach and will take logarithmic time, formally in time complexity $O(\log n)$. As you can imagine, there are endless problems that can be solved using algorithms, and some solutions take longer than others. The time to solve these problems is a function of the number of inputs, which is conventionally denoted by the variable n , and k is some constant.. Time complexities from fastest to slowest include constant time $O(1)$, linear time $O(n)$, logarithmic time $O(\log n)$, polynomial time $O(n^k)$, and exponential time $O(k^n)$. As you might have guessed, a fast solution to problems is preferred in many real-world situations where countless problems are solved via algorithms.

Complexity Classes: P

In computational complexity theory, P and NP refer to classes of problems categorized by the time it takes to solve or verify a solution. The class P (Polynomial Time) includes problems that can be solved in polynomial time, so $O(n^k)$ for some constant $k \leq 1$. Problems in class P have efficient algorithms where the time to compute grows reasonably with input size. In computational complexity theory, Polynomial time is feasible, very good, and considered quick. Examples of problems in class P include sorting algorithms (e.g., merge sort), finding shortest paths (e.g., Dijkstra's algorithm), and primality testing (e.g., AKS Primality Test).

Complexity Classes: NP

The class NP (Nondeterministic Polynomial Time) includes problems for which proposed solutions can be verified efficiently, but finding those solutions may not be efficient. So if someone were to hand you a solution to a problem in the class NP, you could verify if it is correct, but finding the solutions to begin with is not so easy and quick. For instance, given a Sudoku board filled out, checking its correctness is easy, but generating the solution might be hard. The Sudoku problem is an NP problem. Other examples include the Boolean Satisfiability Problem (SAT), the Traveling Salesman Problem, and the Hamiltonian Path Problem. Note that P is a subset of NP; every problem in P is also in NP, because if you can solve a problem fast, you can certainly verify it fast.

The P vs NP Question

So with all the prerequisite knowledge down, what does the problem ask? The P vs NP problem formally asks: Is P equal to NP? In other words, is every problem for which a solution can be verified quickly also a problem for which a solution can be found quickly? Another way to say this is, is it the case that every problem whose solution can be verified quickly (NP) also

one that can be solved quickly (P)? So, is there a quick solution to NP problems that would make them equal to P, having polynomial solutions? A “yes” would imply a massive leap in our computational capabilities and that every problem with a quickly checkable solution also has a quickly findable one. A “no” would confirm that certain problems are inherently hard to solve despite being easy to check.

NP-Completeness and Polynomial-Time Reductions

These next two concepts presented here will bring an even better understanding of the problem and give more detail to the nature of the problem. These two concepts go hand in hand. Within NP lies a subclass known as NP-complete problems; these are the "hardest" problems in NP. A problem is NP complete if it is an NP problem, and every problem in NP can be reduced to it. What does it mean that a problem can be reduced to one another? Let's say you don't know how to solve problem A, but I know how to solve and convert it into problem B, solve that, and convert the solution back. So here, problem A is being reduced to problem B. Stephen Cook formalized and introduced polynomial time reduction as a method for solving one problem using another, by reducing one NP problem into an NP complete problem. This method allows one to transform any problem in NP into an NP-complete problem using a polynomial-time algorithm. In Stephen Cook's seminal paper, he proved the existence of NP-complete problems by proving that the Boolean satisfiability(SAT) problem is NP-complete. He showed that every problem in NP can be translated (reduced) to SAT in polynomial time. Why does NP completeness and reduction matter? Well, this property of NP completeness is important because if any problem in NP can be converted to a problem in NP-Complete, then this would mean that if you find a solution to even one NP-complete problem in polynomial time, you have found a solution for

every problem in NP in polynomial time. Thereby proving that yes, $P = NP$! NP complete problems are the main battleground when trying to solve P vs NP.

Implications of $P = NP$

Why is this question important, and what will its answer mean to the world? Are there any significant changes that will affect our lives? You can imagine that almost everything in our lives uses algorithms; as such, finding a quick solution to hard problems will most definitely have a profound impact. We will explore the case that P is proven to be equal to NP and P is not equal to NP.

The Collapse of Cryptographic Security

Most cryptographic systems rely on the difficulty and hardness of NP problems, they use math problems that are hard to solve but easy to verify. They are so secure because of the fact that NP problems are not feasible to solve. Here are some examples of common cryptographic protocols. Hash functions like SHA-256, which is used in blockchain (Bitcoin mining), are based on the difficulty of inverting the hash function and the difficulty of finding collisions, which is an NP problem. RSA (Rivest-Shamir-Adleman), which is used for secure web browsing (HTTPS), Encrypted emails, and digital signatures, is based on the difficulty of factoring large composite integers into primes, which is an NP problem. ECC (Elliptic Curve Cryptography), which is used for Cryptographic wallets (Bitcoin, Ethereum) and Secure messaging, is based on the Elliptic curve discrete logarithm problem, which is again NP. If $P = NP$, then what was once unfeasible suddenly becomes plausible. What once took 10,000 years of brute-force search...could now be done in 10 seconds.

AI/ Machine Learning

Many bottlenecks in AI are combinatorial and NP-hard, for example, Constraint Satisfaction problems (scheduling, planning), neural architecture search, and probabilistic inference. These problems could be solved efficiently and exactly, and AI could grow in magnitude as compared to the previous years. Natural language processing, image recognition, and decision-making tasks that now require massive computing power or heuristics could be solved in seconds. Entire neural networks could be optimized automatically and instantly.

Biology & Chemistry

Problems like protein folding (how a chain of amino acids folds into a 3D structure) are NP-hard. Drug design involves searching enormous chemical spaces. If $P = NP$, exact protein folding could be solved instantly, drug candidates could be designed in hours instead of years, Genome alignment, evolutionary modeling, and synthetic biology design would become faster and more accurate.

Mathematics

Automated theorem proving, a historically hard task, would become efficient. Proofs that once took years of human effort could be found by machines in minutes. Could generate new theorems and proofs without human input. Mathematics could become machine-driven, producing theorems and proofs far faster than human mathematicians.

Logistics & Operations

Industries like transportation, manufacturing, and supply chain management rely on solving NP-hard optimization problems such as the Traveling Salesman Problem, knapsack, and job scheduling. If $P = NP$, these could be solved optimally in real time, making global workflows faster, cheaper, and more efficient, saving billions of dollars in the process.

Implications of $P \neq NP$

If, on the other hand, P is proven not to equal NP , then: Some problems will remain inherently hard to solve, despite having easily verifiable solutions. Cryptographic systems remain secure. Research continues into approximation algorithms, heuristics, and quantum computing. The theoretical boundary between tractable and intractable problems becomes more clearly defined. Most researchers believe $P \neq NP$ due to the lack of progress despite decades of intense study and the resilience of cryptographic security.

Current Progress and Challenges

Although thousands of problems have been classified as NP -complete, and although SAT has been proven to be NP -complete via the Cook-Levin Theorem, there is still no proof whether P equals NP or not. Significant theoretical barriers remain. One such obstacle is the Natural Proofs Barrier, identified by Razborov and Rudich. Natural proofs are useful in circuit complexity, but are too “general”—if they worked, they would also break cryptographic assumptions. Hence, if cryptography is secure, natural proofs may not resolve P vs NP . As Stephen Cook himself noted, “the sad story is we just don’t know how to proceed.” Despite its age, the P vs NP problem remains as mysterious and as important as ever.

Conclusion

The P vs NP problem is not just a technical question—it is a foundational issue that shapes our understanding of computation, complexity, and feasibility. Whether or not P equals NP , the implications touch nearly every corner of modern science and technology. Until the problem is solved, it will remain a beacon that challenges our ingenuity and defines the limits of what is possible.

References

List of NP-complete problems. (n.d.). *Wikipedia*.

https://en.wikipedia.org/wiki/List_of_NP-complete_problems

NP-completeness. (n.d.). *Wikipedia*. <https://en.wikipedia.org/wiki/NP-completeness>

Many-one reduction. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/Many-one_reduction

Computational complexity theory. (n.d.). *Wikipedia*.

https://en.wikipedia.org/wiki/Computational_complexity_theory

Cook, S. A. (2012, May 2). *Interview with Prof. Stephen A. Cook, 2012 winner of NSERC's \$1M Herzberg Medal* [Video]. YouTube. <https://www.youtube.com/watch?v=q0uAoOccBiw>

Cook, S. A. (2013, August 2). *Cook explains why $P = NP$ matters* [Video]. YouTube. <https://www.youtube.com/watch?v=pQsdygaYcE4>